

Leveraging Machine Learning for Intrusion Detection and Classification

Mohammad Malkawi

mialmalkawi22@cit.just.edu.jo

Abstract—This work describes a performance for seven supervised machine learning algorithms on intrusion detection on the ToN-IoT Train/Test Network dataset. After removing duplicates and nominal attribute encoding, traditional classifiers (Naive Bayesian, Logistic Regression, Decision Trees, RF, and RBF SVM) and ensemble models (Bagging, AdaBoost) were compared for intrusion detection performance on a stratified 75/25 split of the dataset, showing that AdaBoost had 100% accuracy, precision, recall, and F1-score on this task, with Decision Trees and RF classifiers also performing very closely.

Keywords—intrusion detection, ToN-IoT, network traffic, machine learning, ensemble learning

I. INTRODUCTION

(IDS tools are essential in securing the current state of IoT networks because of the high amounts of traffic and attacks experienced in these networks. ML is an alternative because it is data-driven and learns patterns to differentiate between malicious and legal traffic.

This report aims to implement and compare a variety of supervised learning models for the binary intrusion detection task using the ToN-IoT Train/Test Network dataset. This report seeks to analyze a common IDS task: (1) the pre-processing of network flow features, (2) the training of classifiers, and (3) the assessment of accuracy, precision, recall, F1, and cost, as is required by the course project.

II. METHODOLOGY

A. Dataset

The used Dataset “TON-IOT”, contains numerical and categorical features and classification labels, Missing values were handled using Forward Fill, and duplicated records were removed to make sure that all data are clean.

featuring multiple behavioral parameters like protocol, DNS, SSL, HTTP, among others, along with two ground truth markings, namely a binary mark (normal vs. attack) and attack type for multi-class evaluation [1], [3]. The CSV file (train_test_network.csv) contains 211,043 records with 44 columns (43 feature columns, including the label/type variables). The binary label is imbalanced, where there are 161,043 attack records, while normal records are 50,000. There are nine malicious classes, including the normal class, where the 'mitm' class is the smallest. The dataset is also available online at Kaggle.

C. Model's implemented

Several supervised learning algorithms were tested using scikit-learn [2]. All algorithms were trained on the standardized set for training, with the test set held aside. Results include Accuracy, Precision, Recall, F1 score for the positive class (attack=1), as well as model training times as done in the included notebook.

1. NAIVE BAYES
2. LOGISTIC REGRESSION (MAX_ITER=1000)
3. DECISION TREE (CART)
4. RANDOM FOREST (100 TREES)
5. SUPPORT VECTOR MACHINE WITH RBF KERNEL
6. BAGGING (50 DECISION-TREE BASE ESTIMATORS)
7. ADABOOST (50 ESTIMATORS)

B. Abbreviations and Acronyms

IDS: Intrusion Detection System.

IoT: Internet of Things.

ToN-IoT: Telemetry of Network–Internet of Things

ML: Machine Learning.

NB: Naïve Bayes

LR: Logistic Regression.

DT: Decision Tree.

RF: Random Forest.

SVM: Support Vector Machine.

Bagging: Bootstrap Aggregating ensemble method

AdaBoost: Adaptive Boosting

TP: True Positive.

TN: True Negative.

FP: False Positive.

FN: False Negative.

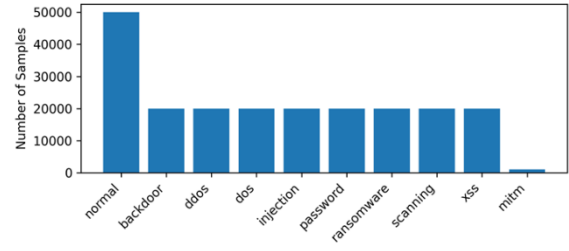


Fig2. Distribution of attack types in the dataset

C. Preprocessing

Categorical features were encoded using label encoding, and the numerical features were normalized using standard scaling.

The preprocess steps follow the implementation in the notebook. First, we handled duplicate rows by removing exact duplicates. The data does not have any missing values. Then, we filtered columns with "ip" in them to avoid leakage from Irvine datasets with actual IPs. This resulted in removing 5 columns: <src_ip, dst_ip, src_ip_bytes, dst_ip_bytes, ssl_cipher. Next, Label Encoding was performed on object columns, and z-scoring was used to standardize all features. A 75/25 stratified split was used to split the data into training 80% and testing 20% .

Dataset summary

Item	Value
Record (before cleaning)	211,043
Duplicate records removed	20,569
Record (after cleaning)	190,474
Total columns	44(43features+lable/type)
Binary classes (label)	Normal =50,000 Attack =161,043
Attack type	Normal scanning, dos,ddos,injection ,password,xss,ransomware,backdoor,mitm

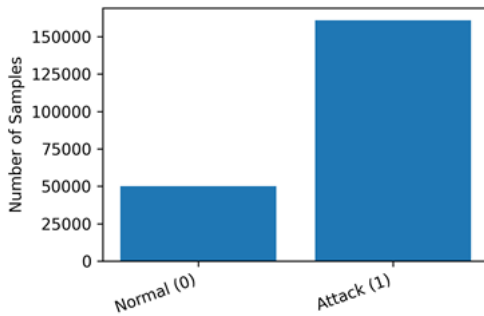


Fig1.class distribution for binary label (Normal(0) vs Attack(1))

D. Results and analysis

Table II compares the quantitative performance of all models assessed. On general, the ensemble algorithms and tree classifiers demonstrate exceptionally strong detection capability on this data. AdaBoost algorithm scored a perfect mark on all parameters in the testing subset.

Evaluation Metrics : Performance was evaluated using: Accuracy , Precision , Recall , F1-Score , Inference Time.

	Accuracy	Precision	Recall	F1-Score	Inference Time
Naive Bayes	0.843949	0.833161	1.000000	0.908988	0.052596
Logistic Regression	0.984103	0.989708	0.989895	0.989801	0.331697
Decision Tree	0.999685	0.999865	0.999731	0.999798	0.613670
Random Forest	0.999916	0.999946	0.999946	0.999946	5.499033
SVM	0.994498	0.994604	0.998356	0.996477	20.519395
Bagging	0.999748	0.999865	0.999811	0.999838	17.889268
AdaBoost	1.000000	1.000000	1.000000	1.000000	2.899638

TABLE II.MODEL PERFORMANCE ON THE HELD-OUT TEST SET

Naive Bayes gives high recall but relatively low precision and accuracy, which indicates that this method is pointing towards the major attacks but with higher false positives. Logistic Regression and SVM give very good results with moderate training cost. Decision Trees and RF give almost perfect results in accuracy, which indicates that the patterns in the dataset are very separable for attack identification. Bagging makes the model robust but adds to the cost because of the multiple models to be trained. AdaBoost gives the best results with a reasonable training cost compared to the previous two methods: RF and SVM.

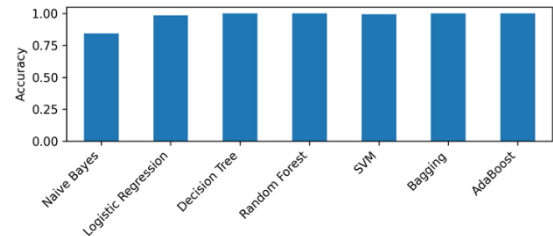


Fig. 3.Accuracy comparison across evaluated models

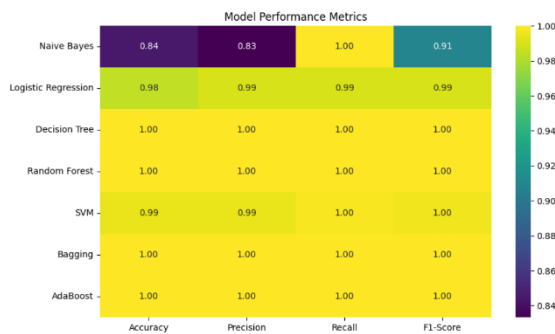


Fig. 4. Heatmap of core evaluation metrics (Accuracy, Precision, Recall, F1-score).

The outcome demonstrates that traditional ML models have proven very successful in intrusion detection tasks for ToN-IoT network flow attributes. Yet, some aspects could lead to an exaggeration of the performance metric prediction. Firstly, this analysis uses one hold-out split in a random manner, which could involve very correlated flows in the data set. Secondly, Label Encoding assigns an unnatural order to categorical information, which could be done better by employing target/hashing encoders: one-hot encoding could preserve more accurate information. Lastly, this process excludes all columns that include "ip" in them, which could include meaningful fields like ssl_cipher, due to name overlap, which needs correction in a final application.

In conclusion, this study concentrates on binary detection tasks (normal or attack). The attack type information available within the dataset can be used to allow multi-class classification. The models could be extended to a multi-class scenario to test on separate data within a temporal context.

CONCLUSION AND FUTURE WORK

This experiment involved the use of a ToN-IoT Train/Test Network ML task to detect intrusions. After removing the duplicates, coding the variables, scaling the variables, and creating a stratified split ratio of 75% and 25%, the results achieved by the classification algorithms were determined.

AdaBoost scored a perfect test set result, while Decision Tree and Random Forest scored a near-perfect test set result.

Future tasks will include: (1) carrying out multi-class attack classification with the 'type' label, (2) using stronger category encoding and feature selection techniques, (3) testing the inference generalization with cross-validation and time-based splits, and (4) calculating the actual inference time for deployment-related testing.

Evaluation metrics

The performance is measured by Accuracy, Precision, Recall, F1-Score, and Inference Time. The Accuracy measures overall correctness, while Precision and Recall depict the relationship between false alarms and missed attacks on the intrusion-detection system. The F1-Score is a combination of Precision and Recall. The Inference Time measures the average time required by the model for making predictions while testing. It is useful in real-time intrusion detection systems.

REFERENCES

- [1] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustainable Cities and Society*, vol. 72, Art. no. 102994, 2021.
- [2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] UNSW Canberra at ADFA, "The TON_IoT datasets (dataset portal)," UNSW Research, accessed 2025.
- [4] A. Bhowmik, "ToN-IoT Network Dataset," Kaggle, accessed 2025. [Online]. Available: <https://www.kaggle.com/datasets/arnobbbhowmik/ton-iot-network-dataset>

Link for Jupyter notebooks:

<https://anaconda.com/app/share/notebooks/ec80f42b-9594-4d20-911fc7501d2688e3/overview>

Link for the dataset used:

<https://www.kaggle.com/datasets/arnobbbhowmik/ton-iot-network-dataset>